

Redes Neuronales para predicción

Las redes neuronales siempre han sido uno de los modelos de aprendizaje automático más fascinantes, no solo por el sofisticado algoritmo de retropropagación, sino también por su complejidad (piensa en el aprendizaje profundo con muchas capas ocultas) y la estructura inspirada en el cerebro.

Las redes neuronales no siempre han sido populares, en parte porque eran, y aún en algunos casos son, computacionalmente costosas y en parte porque **no parecen producir mejores resultados en comparación con métodos más simples como máquinas de vectores de soporte (SVM)**. Sin embargo, Neural Networks, una vez más, ha llamado la atención y se ha hecho popular.

El conjunto de datos

Vamos a usar el conjunto de datos de Boston en el paquete MASS.

El conjunto de datos de Boston es una recopilación de datos sobre el valor de la vivienda en los suburbios de Boston. Nuestro objetivo es predecir el valor medio de las viviendas ocupadas por el propietario (medv) utilizando todas las demás variables continuas disponibles.

```
> set.seed(500)
> library(MASS)
> data <- Boston
```

Primero, debemos verificar que no falte ningún punto de datos; de lo contrario, debemos corregir el conjunto de datos

```
> apply(data,2,function(x) sum(is.na(x)))
  crim    zn  indus  chas  nox   rm   age   dis   rad   tax
  0     0    0     0    0    0    0    0    0    0
pratio black  lstat  medv
  0     0    0     0
```

No hay datos faltantes, bien. Procedemos dividiendo aleatoriamente los datos en un conjunto de entrenamiento y de prueba, luego ajustamos un modelo de regresión lineal y lo probamos en el conjunto de prueba. Ten en cuenta que **estamos usando la función glm() en lugar de lm()**, que será útil más adelante cuando se valide de forma cruzada el modelo lineal.

```
> index <- sample(1:nrow(data),round(0.75*nrow(data)))
> train <- data[index,]
> test <- data[-index,]
> lm.fit <- glm(medv~., data=train)
> summary(lm.fit)
```

Call:

```
glm(formula = medv ~ ., data = train)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-14.9143	-2.8607	-0.5244	1.5242	25.0004

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	43.469681	6.099347	7.127	5.50e-12 ***
crim	-0.105439	0.057095	-1.847	0.065596 .
zn	0.044347	0.015974	2.776	0.005782 **
indus	0.024034	0.071107	0.338	0.735556
chas	2.596028	1.089369	2.383	0.017679 *
nox	-22.336623	4.572254	-4.885	1.55e-06 ***
rm	3.538957	0.472374	7.492	5.15e-13 ***
age	0.016976	0.015088	1.125	0.261291
dis	-1.570970	0.235280	-6.677	9.07e-11 ***
rad	0.400502	0.085475	4.686	3.94e-06 ***
tax	-0.015165	0.004599	-3.297	0.001072 **
ptratio	-1.147046	0.155702	-7.367	1.17e-12 ***
black	0.010338	0.003077	3.360	0.000862 ***
lstat	-0.524957	0.056899	-9.226	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for gaussian family taken to be 23.26491)

Null deviance: 33642 on 379 degrees of freedom
Residual deviance: 8515 on 366 degrees of freedom
AIC: 2290

Number of Fisher Scoring iterations: 2

```
> pr.lm <- predict(lm.fit,test)
> MSE.lm <- sum((pr.lm - test$medv)^2)/nrow(test)
```

La función de muestra (x, size) simplemente genera un vector del tamaño especificado de muestras seleccionadas aleatoriamente del vector x. Por defecto, el muestreo no tiene reemplazo: el índice es esencialmente un vector aleatorio de índices.

Ya que estamos lidiando con un problema de regresión, vamos a usar el error medio cuadrático (MSE) como una medida de cuánto nuestras predicciones están muy lejos de los datos reales.

Preparación para adaptarse a la red neuronal

Antes de utilizar una red neuronal, se debe hacer algo de preparación. Las redes neuronales no son tan fáciles de entrenar y sintonizar.

Como **primer paso**, vamos a **abordar el preprocesamiento de datos**.

Es una buena práctica **normalizar tus datos antes de entrenar una red neuronal**. No puedo enfatizar suficientemente cuán importante es este paso: **dependiendo de tu conjunto de datos, evitar la normalización puede llevar a resultados inútiles o a un proceso de entrenamiento muy difícil** (la mayoría de las veces el algoritmo no convergerá antes de la cantidad máxima de iteraciones permitidas). Puedes elegir diferentes métodos para escalar los datos (z-normalización, escala min-max, etc.). Elegimos usar el **método min-max** y **escalar los datos en el intervalo [0,1]**. Por lo general, **la escala en los intervalos [0,1] o [-1,1] tiende a dar mejores resultados**.

Por lo tanto, escalamos y dividimos los datos antes de continuar:

```
> maxs <- apply(data, 2, max)
> mins <- apply(data, 2, min)
> scaled <- as.data.frame(scale(data, center = mins, scale = maxs - mins))
> train_ <- scaled[index,]
> test_ <- scaled[-index,]
```

Ten en cuenta que **la escala devuelve una matriz que necesita ser coaccionada en un data.frame**.

Parámetros

Por lo que sabemos, **no hay una regla fija en cuanto a la cantidad de capas y neuronas para usar**, aunque hay varias reglas generales más o menos aceptadas. Por lo general, si es necesario, **una capa oculta es suficiente para un gran número de aplicaciones**. En lo que respecta al número de neuronas, **debe estar entre el tamaño de la capa de entrada y el tamaño de la capa de salida**, por lo general **2/3 del tamaño de la entrada**. Al menos en mi experiencia, probar una y otra vez es la mejor solución, ya que no hay garantía de que alguna de estas reglas se ajuste mejor a tu modelo.

Dado que este es un ejemplo de juguete, vamos a utilizar 2 capas ocultas con esta **configuración: 13: 5: 3: 1**. La capa de entrada tiene 13 entradas, las dos capas ocultas tienen 5 y 3 neuronas y la capa de salida tiene, por supuesto, una salida única, ya que estamos haciendo la regresión.

Vamos a ajustar la red:

```
> library(neuralnet)
> n <- names(train_)
> f <- as.formula(paste("medv ~", paste(n[!n %in% "medv"], collapse = " + ")))
> nn <- neuralnet(f,data=train_,hidden=c(5,3),linear.output=T)
```

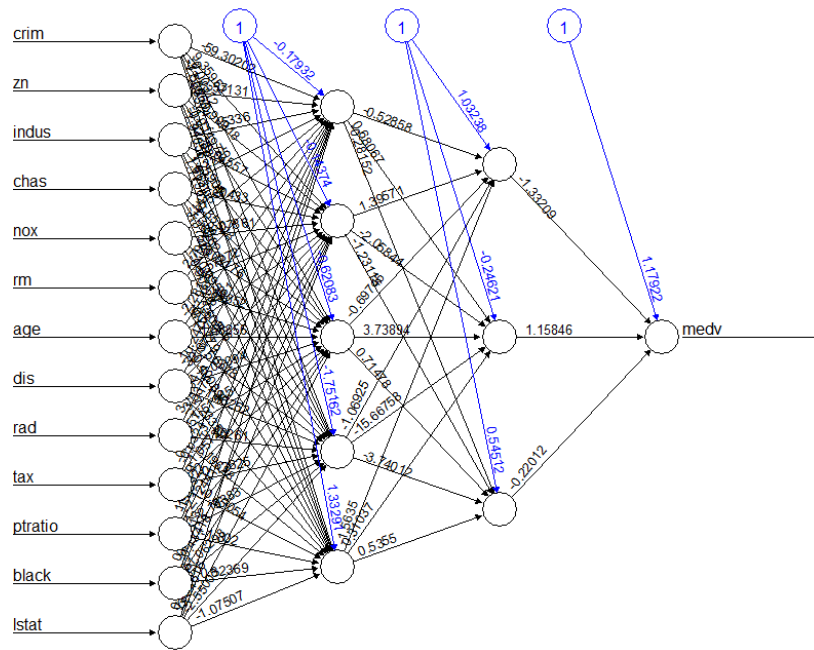
Un par de notas:

- Por alguna razón, la **fórmula y ~.** no se acepta en la función `neuralnet()`. Primero debes escribir la fórmula y luego pasarla como argumento en la función de ajuste.
- **El argumento oculto acepta un vector con el número de neuronas para cada capa oculta**, mientras que el argumento **linear.output** se usa para especificar si queremos hacer **regresión** `linear.output = TRUE` o **classification** `linear.output = FALSE`

El paquete neuralnet proporciona una buena herramienta para trazar el modelo:

```
> plot(nn)
```

Esta es la representación gráfica del modelo con los pesos en cada conexión:



Las líneas negras muestran las conexiones entre cada capa y los pesos en cada conexión, mientras que las líneas azules muestran el término de sesgo agregado en cada paso. El sesgo puede considerarse como la intersección de un modelo lineal.

La red es esencialmente una caja negra, por lo que no podemos decir mucho sobre el ajuste, los pesos y el modelo. Basta decir que el algoritmo de entrenamiento ha convergido y, por lo tanto, el modelo está listo para ser utilizado.

Predecir medv usando la red neuronal

Ahora podemos intentar predecir los valores para el conjunto de prueba y calcular el MSE. Recuerda que la red producirá una predicción normalizada, por lo que debemos reducirla para hacer una comparación significativa (o simplemente una predicción simple).

```
> pr.nn <- compute(nn,test[,1:13])
> pr.nn_ <- pr.nn$net.result*(max(data$medv)-min(data$medv))+min(data$medv)
> test.r <- (test_$medv)*(max(data$medv)-min(data$medv))+min(data$medv)
> MSE.nn <- sum((test.r - pr.nn_)^2)/nrow(test_)
```

luego comparamos las dos MSE

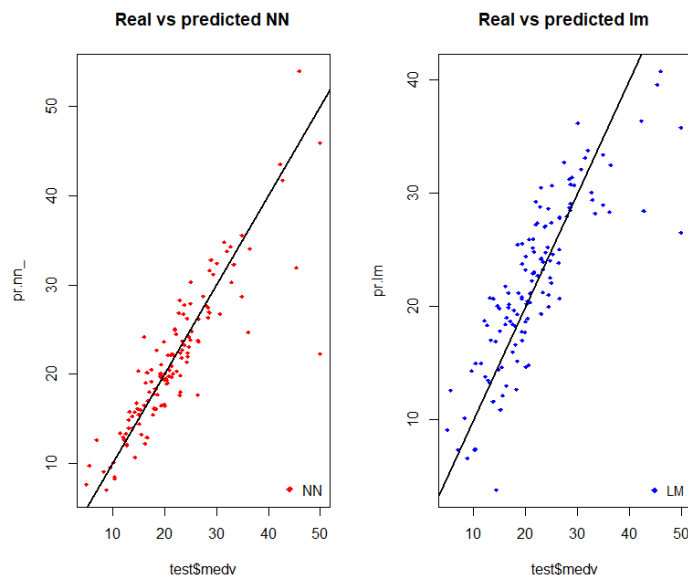
```
> print(paste(MSE.lm,MSE.nn))  
[1] "21.6297593507225 15.7518370200153"
```

Aparentemente, la red está haciendo un mejor trabajo que el modelo lineal para predecir medv. Una vez más, **ten cuidado porque este resultado depende de la división de prueba de entrenamiento realizada anteriormente**. A continuación, después de la parte visual (plot), vamos a realizar una validación cruzada rápida para tener más confianza en los resultados.

Un primer enfoque visual para el rendimiento de la red y el modelo lineal en el conjunto de prueba se muestra a continuación

```
> par(mfrow=c(1,2))  
> plot(test$medv,pr.nn_,col='red',main='Real vs predicted NN',pch=18,cex=0.7)  
> abline(0,1,lwd=2)  
> legend('bottomright',legend='NN',pch=18,col='red', bty='n')  
> plot(test$medv,pr.lm,col='blue',main='Real vs predicted lm',pch=18, cex=0.7)  
> abline(0,1,lwd=2)  
> legend('bottomright',legend='LM',pch=18,col='blue', bty='n', cex=.95)
```

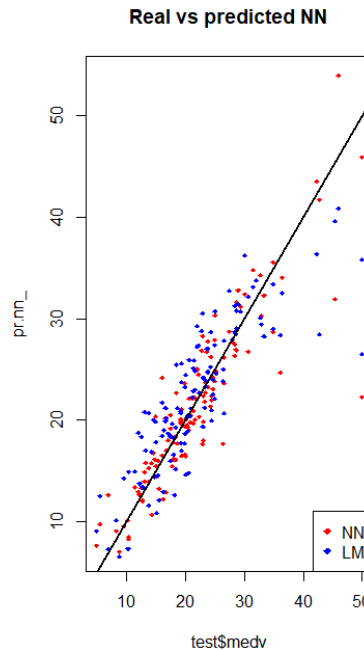
El diagrama de salida



Al inspeccionar visualmente el plot, podemos ver que las predicciones hechas por la red neuronal están (en general) más concentradas alrededor de la línea **(una alineación perfecta con la línea indicaría una MSE de 0 y por lo tanto una predicción perfecta)** que las hechas por el modelo.

```
> plot(test$medv,pr.nn_,col='red',main='Real vs predicted NN',pch=18,cex=0.7)  
> points(test$medv,pr.lm,col='blue',pch=18,cex=0.7)  
> abline(0,1,lwd=2)  
> legend('bottomright',legend=c('NN','LM'),pch=18,col=c('red','blue'))
```

Una comparación visual quizás más útil se traza a continuación:



Una validación cruzada (rápida)

La validación cruzada es otro paso muy importante de la construcción de modelos predictivos. Si bien hay diferentes tipos de métodos de validación cruzada, la idea básica es repetir el siguiente proceso una cantidad de tiempo:

prueba de tren dividida

- Haz la prueba de tren dividida
- Ajusta el modelo al conjunto de entrenamiento
- Prueba el modelo en el conjunto de prueba
- Calcula el error de predicción
- Repite el proceso K veces

Luego, al calcular el error promedio, podemos comprender cómo funciona el modelo.

Vamos a implementar una validación cruzada rápida utilizando un ciclo for para la red neuronal y la función `cv.glm()` en el paquete de arranque para el modelo lineal. Por lo que sabemos, no existe una función incorporada en R para realizar la validación cruzada en este tipo de red neuronal. Aquí está el MSE 10 veces cruzado validado para el modelo lineal:

```
> install.packages("boot")  
> library(boot)  
> set.seed(200)
```

```
> lm.fit <- glm(medv~.,data=data)
> cv.glm(data,lm.fit,K=10)$delta[1]
[1] 23.83560156
```

Ahora la red. Ten en cuenta que **estamos dividiendo los datos** de esta manera: **90%** conjunto de entrenamiento y **10%** de prueba **establecidos de forma aleatoria por 10 veces**. También estamos inicializando una barra de progreso usando la biblioteca plyr porque queremos estar al tanto del estado del proceso ya que la instalación de la red neuronal puede tardar un tiempo.

```
> install.packages("plyr")
set.seed(450)
cv.error <- NULL
k <- 10

library(plyr)
pbar <- create_progress_bar('text')
pbar$init(k)

for(i in 1:k){
  index <- sample(1:nrow(data),round(0.9*nrow(data)))
  train.cv <- scaled[index,]
  test.cv <- scaled[-index,]

  nn <- neuralnet(f,data=train.cv,hidden=c(5,2),linear.output=T)

  pr.nn <- compute(nn,test.cv[,1:13])
  pr.nn <- pr.nn$net.result*(max(data$medv)-min(data$medv))+min(data$medv)

  test.cv.r <- (test.cv$medv)*(max(data$medv)-min(data$medv))+min(data$medv)

  cv.error[i] <- sum((test.cv.r - pr.nn)^2)/nrow(test.cv)

  pbar$step()
}
```

Después de un tiempo, el proceso está hecho, calculamos el promedio de MSE y trazamos los resultados como un diagrama de caja (boxplot).

```
mean(cv.error)
10.32697995

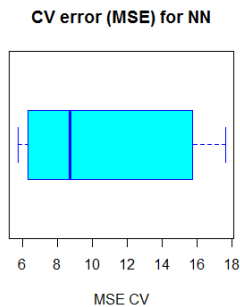
cv.error
17.640652805 6.310575067 15.769518577 5.730130820 10.520947119 6.121160840
6.389967211 8.004786424 17.369282494 9.412778105
```

El código para el diagrama de caja:

```
boxplot(cv.error,xlab='MSE CV',col='cyan',
```

```
border='blue',names='CV error (MSE)',  
main='CV error (MSE) for NN',horizontal=TRUE)
```

El código anterior muestra el siguiente diagrama de caja:



Como puedes ver, el promedio de MSE para la red neuronal (10.33) es menor que el del modelo lineal, aunque parece haber un cierto grado de variación en las MSE de la validación cruzada. Esto puede depender de la división de los datos o la inicialización aleatoria de los pesos en la red. Al ejecutar la simulación en diferentes momentos con diferentes semillas, puedes obtener una estimación puntual más precisa para la MSE promedio.

Una nota final sobre la interpretabilidad del modelo

Las redes neuronales se parecen mucho a las cajas negras: explicar su resultado es mucho más difícil que explicar el resultado de un modelo más simple, como un modelo lineal. Por lo tanto, dependiendo del tipo de aplicación que necesites, es posible que desees tener en cuenta este factor también. Además, como has visto anteriormente, se necesita cuidado adicional para adaptarse a una red neuronal y pequeños cambios pueden conducir a resultados diferentes.